



Centro de
Informática
UFPE

LLM-Based Test-Driven Interactive Code Generation

User Study and Empirical Evaluation

Matheus Borges <mbf3> Mikael Cavalcanti <mcs11> Pedro Domingues <prds>

Motivação

- LLMs são ótimos para gerar código a partir de "Linguagem Natural" (NL).
- A Linguagem Natural é informal, ambígua e incompleta. Ela não é uma "especificação" formal.
- Desenvolvedores podem aceitar códigos gerados por IA que "parecem" corretos, mas contêm bugs sutis por não possuir validação.
- Um novo **fluxo de trabalho** interativo que esclarece a intenção do usuário antes de finalizar o código (TICODER).

```
1 def text_lowercase_underscore(text):  
2     """Write a function that returns true if the  
    input string contains sequences of lowercase  
    letters joined with an underscore and false  
    otherwise"""
```

Introdução

- LLMs como o GitHub Copilot são populares, mas têm desafios.
- A linguagem natural (NL) não captura totalmente a intenção do usuário.
- Inconsistência nas saídas da função / código.
- Problemas como lista de sugestões.
- Grande carga cognitiva do desenvolvedor para avaliação dos códigos.

TICODER

- test-driven interactive code generation
- Esclarecer (formalizar) a intenção do usuário de através de **testes**.
- Gerar lista de códigos semanticamente consistentes com os testes.

```
1 def text_lowercase_underscore(text):  
2     """Write a function that returns true if the  
    input string contains sequences of lowercase  
    letters joined with an underscore and false  
    otherwise"""
```

1

```
text_lowercase_underscore("aa_bb_cc") == True?
```

2

```
text_lowercase_underscore("aa_bb") == True?
```

```
def text_lowercase_underscore(text):  
    return True if bool(re.search(r'^[a-z]_[a-z]+$  
, text)) else False  
def test_text_lowercase_underscore():  
    assert text_lowercase_underscore("aa_bb")==  
        True  
test_text_lowercase_underscore()
```

Entendendo as limitações

- Não escalabilidade devido a complexidade de alguns testes e fluxo de trabalho.
- Depende de Estrutura:
 - LLM capaz de gerar teste úteis.
 - Relação de custo-benefício entre grande interação do usuário e benefício na eliminação e sugestão de código.

Trabalhos relacionados

- Comparação em dois pontos:
 - Melhorando a precisão da geração do código.
 - Usabilidades dos assistentes de programação

Precisão na geração de código

- AlphaCode
 - Gera testes usando LLM e agrupa as sugestões de códigos que satisfazem os testes.
- CodeT
 - Refina a abordagem apontando testes e sugestões.
 - Agrupa sugestões que satisfazem mais testes.
 - Agrupa testes que se encaixa e mais sugestões.
 - Se assemelha com TICODER por usar testes gerados por LLM para reclassificar código gerado.
 - TICODER pode ser utilizado como algo complementar
- TICODER permite modificar as saídas dos testes oque o CodeT não permite.
- TICODER diferencia por utilizar especificações informais (intenção em linguagem natural).

Usabilidade dos assistentes

- Pontos abordados em outros trabalhos:
 - Dificuldade de incorporar o código gerado e falta de fornecer feedback faz as pessoas desistirem do uso.
 - Eliminação de ambiguidade para assistentes de código como Github Copilot.
 - Dificuldade do desenvolvedor em expressar as intenções em linguagem natural.
 - Programadores iteram diversas vezes na geração de código e precisam se desgastar revisando os códigos.

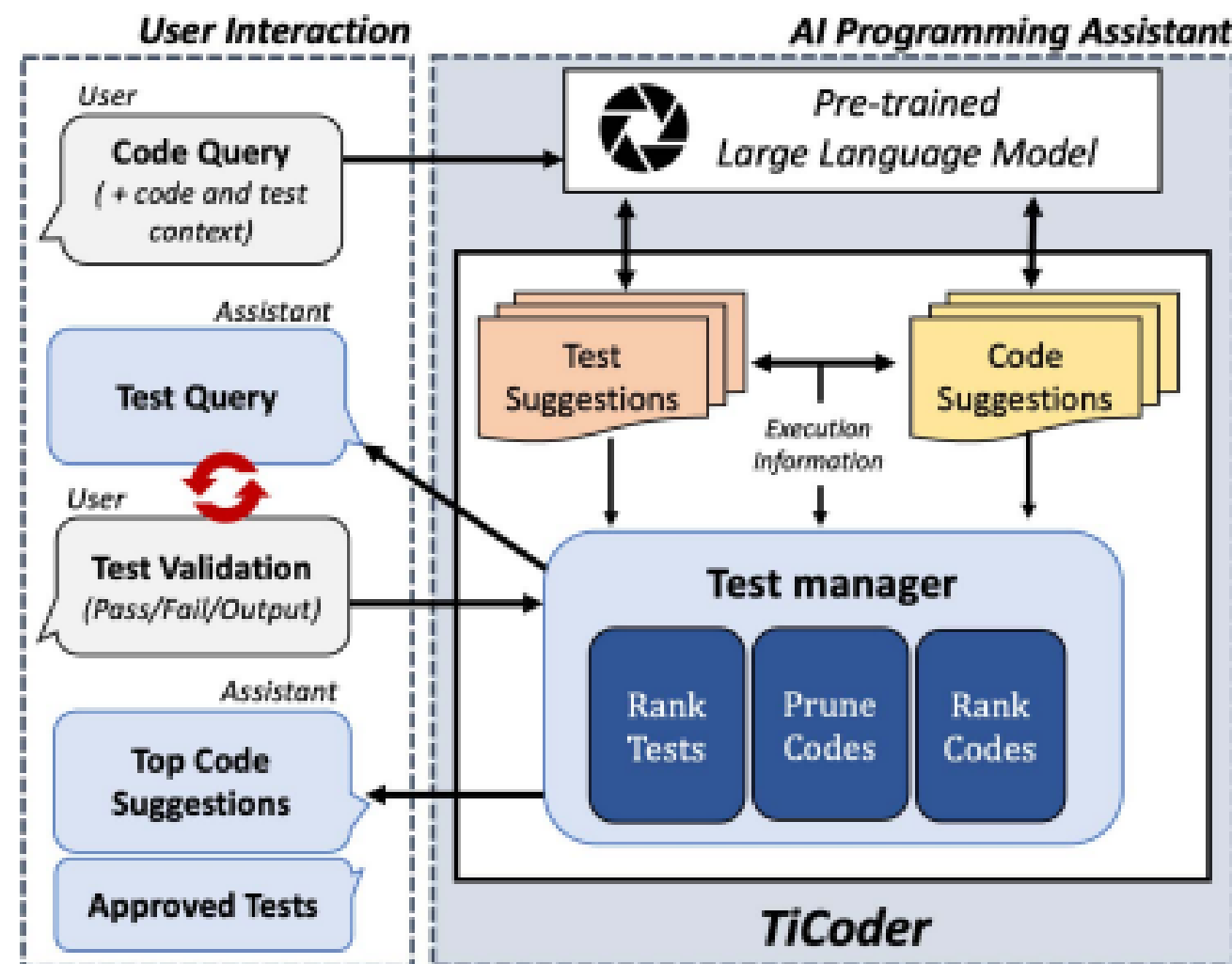
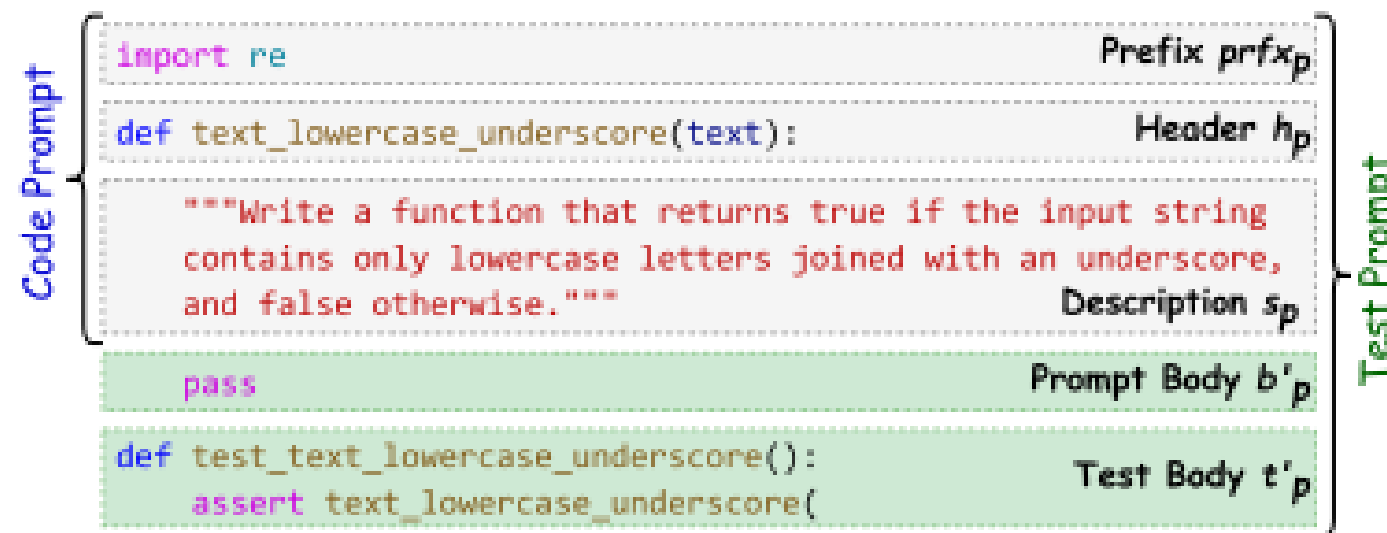
Questões de pesquisa e organização do artigo

- Como o fluxo de trabalho proposto afeta o desempenho dos desenvolvedores que avaliam o código gerado por IA?
- O fluxo de trabalho proposto aumenta a precisão dos modelos de geração de código?
- Principais pontos explorados para responder as perguntas:
 - Estudo dos desenvolvedores utilizando o fluxo de trabalho e avaliando a dificuldade de avaliação dos códigos
 - Avaliação da abordagem através de benchmarks

Abordagem Proposta: TICODER

- Fluxo de trabalho de alto nível
- Implementação do TICODER

Fluxo de trabalho de alto nível



- **TICODER-PASSFAIL**
 - O desenvolvedor fornece uma resposta boolean (PASS ou FAIL).
 - Exige mais da LLM.
- **TICODER-OUTPUT**
 - O desenvolvedor fornece a saída correta para o teste que falhar.
 - Pode exigir um oráculos de teste não trivial, além de prover entradas e saídas.
 - Exige mais do desenvolvedor.

Implementação do TICODER

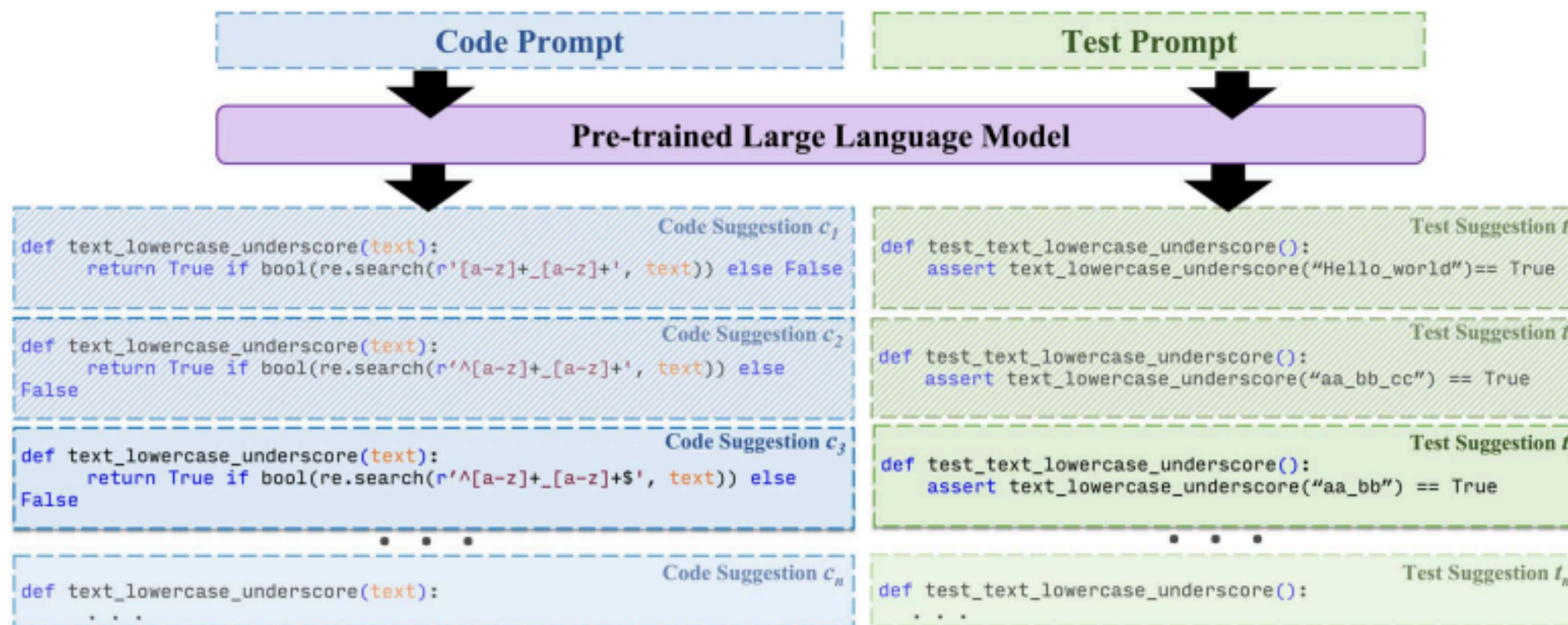


Fig. 3. Code and test suggestions for the running example in Fig. 2 generated from a LLM. Code suggestion c_3 and test suggestion t_3 are both *correct*, while code suggestions c_1, c_2 and test suggestions t_1, t_2 are *incorrect* (appear shaded), i.e. they don't satisfy the *problem prompts* in Fig. 2.

$$s_{discr}(t) = \begin{cases} 0 & \text{if } \max(|G_t^+|, |G_t^-|) \text{ is } 0 \\ \min(|G_t^+|, |G_t^-|) / \max(|G_t^+|, |G_t^-|) & \text{otherwise} \end{cases}$$

- Gerando código e teste.
 - Através da interação / prompt do usuário.
- Classificação das sugestões de teste.
 - TICODER-PASSFAIL e TICODER-OUTPUT, usado para podar as classificações de sugestão de código.
- Poda e classifica a sugestão de código.
 - Usuário responde PASS (aprova), Fail (corta da lista de sugestão).
 - Devolução do OUTPUT serve para podar ainda mais as sugestões.

Vantagens:

- **Feedback Verificável**
- **Filtragem prévia**
- **Intenção formalizada**

Limitações:

- **Qualidade dos testes**
- **Testes complexos podem aumentar carga cognitiva**
- **Usuário ainda precisa entender o que está sendo feito**

Participantes

- 8 Profissionais da indústria
- 10 Estudantes de PhD

ID	Python Experience	Python Frequency	AI Programming Assistant Use	Occupation
Pilot	>5 years	Daily	Daily	Industry
Pilot	>5 years	Monthly	Monthly	Industry
Pilot	>5 years	Daily	Daily	Industry
P1	>5 years	Monthly	Daily	Industry
P2	>5 years	Weekly	Monthly	Industry
P3	>5 years	Rarely or never	Rarely or never	Industry
P4	>5 years	Daily	Daily	Industry
P5	3 - 5 years	Weekly	Weekly	Academia
P6	>5 years	Weekly	Weekly	Academia
P7	>5 years	Weekly	Monthly	Industry
P8	>5 years	Monthly	Rarely or never	Academia
P9	>5 years	Daily	Monthly	Academia
P10	1 - 2 years	Weekly	Daily	Academia
P11	>5 years	Daily	Daily	Academia
P12	3 - 5 years	Weekly	Rarely or never	Academia
P13	>5 years	Weekly	Rarely or never	Academia
P14	3 - 5 years	Rarely or never	Rarely or never	Academia
P15	>5 years	Daily	Daily	Industry

Assistente 1 (Controle)

5 sugestões de
código em
ordem aleatória

Assistente 2 (TICODER-PASSFAIL) (TICODER-OUTPUT)

5 sugestões
geradas, mostra em
ordem aleatória as
que passam nos
testes

5 sugestões
geradas, mostra em
ordem aleatória as
que passam nos
testes

Tarefas selecionadas

Task	Task Name	Description	Treatments
T1	LOWERUNDERSCORE	Write a function that returns true if the input string consists of two sequences of lowercase letters joined with a single underscore and false otherwise.	A1, A2, A3
T2	FIRSTMISSING	Write a function that finds the smallest missing number from a sorted list of integers, starting from 0.	A1, A2, A3
T3	MAXPRODUCT	Write a function to find the maximum product formed by multiplying numbers of an increasing contiguous subsequence of that array. The sequence may include negative numbers.	A1, A2, A3

Exemplos das tarefas com o assistente 1, 2 e 3 respectivamente.

You want to generate a python function using the following prompt:

Write a function that returns true if the input string consists of two sequences of lowercase letters joined with a single underscore and false otherwise.

Using the prompt above, AI Assistant #1 generates the following 5 code suggestions:

Suggestion 1:

```
def text_lowercase_underscore(s):  
    return True if any([s[i:i+2].islower() and s[i+2:i+3]==_ for i in range(len(s)-2)]) else False
```

Suggestion 2:

```
def text_lowercase_underscore(string):  
    return bool(re.match(r'^[a-z]{1,20}_[a-z]{1,20}$', string))
```

You want to generate a python function using the following prompt:

Write a python function to find the smallest missing number from a sorted list of integers, starting from 0.

Before generating code suggestions, AI Assistant #2 wants to ask you a few questions:

Should the following assertion pass or fail?

```
assert find_First_Missing([1,2,4,6]) == 0
```

PASS

FAIL

You want to generate a python function using the following prompt:

Write a function to find the maximum product formed by multiplying numbers of an increasing contiguous subsequence of that array. The sequence may include negative numbers.

Before generating code suggestions, AI Assistant #3 wants to ask you a few questions:

```
assert max_product([3, 4, 2]) == ?
```

Metric	Assistant 1 (mean)	Assistant 2 (mean)	Assistant 3 (mean)	Pairwise Significance
Correctness* (0,1)	0.40	0.84	0.64	$a1 < a2 (p = 0.001)$
Time (seconds)	327.7	284.15	253.88	-
Cognitive Load* (0-100)	45.46	28.00	29.52	$a1 > a2 (p = 0.007)$ $a1 > a3 (p = 0.012)$
Mental Demand* (0-20)	12.5	7.50	7.6	$a1 > a2 (p = 0.001)$ $a1 > a3 (p = 0.004)$
Stress* (0-20)	8.26	3.84	6.35	$a1 > a2 (p = 0.02)$
Pace* (0-20)	8.13	5.38	4.70	$a1 > a3 (p = 0.04)$
Confidence (0-20)	13.5	15.92	15.88	-
Effort* (0-20)	11.00	7.15	6.7	$a1 > a2 (p = 0.02)$ $a1 > a3 (p = 0.014)$

- Corretude
- Carga cognitiva
- Demanda Mental
- Estresse
- Ritmo
- Esforço

Benchmarking com LLMs

*Bases: MBPP e HumanEval;
pass@1@m, Probabilidade do primeiro código gerado
ser correto após m iterações de feedback*

Fluxo TICODER - Ciclo Iterativo

- Entrada: Prompt em linguagem natural
- Processo:
 - LLM gera código + testes unitários
 - Execução automática dos testes
 - Feedback do Oracle (simulação perfeita)
 - Filtragem (pruning) de código inconsistente
 - Repete por 1-2 iterações (m)
- Saída: Código validado + testes aprovados

PORTFÓLIO DE LLMs TESTADOS

Quatro modelos com características distintas:

- Nome (Parâmetros, Especialização, Custo)
- CodeGen-6B (6B, código, 1x)
- CodeT (~16B, código+testes, 3x)
- GPT-3.5 (175B, generalista, 10x)
- GPT-4 (~1.7T, SotA, 50x)

.

Dados!

- Melhora média de 45% na acurácia após iterações
- Modelos pequenos chegando perto da performance do GPT-4
- Convergência rápida (1–2 iterações)
- Limitação crítica:
 - O 'usuário' é um oráculo perfeito. Na vida real... humanos erram, se distraem, interpretam mal. Este é o teto teórico.

Resultados Escaláveis

Em avaliação em larga escala com MBPP e HumanEval, o TICODER alcançou melhora média de +45% na acurácia métrica pass@1@m após poucas iterações

Contribuições

- ✓ Nova forma de colaboração humano-IA
 - ✓ Formalização parcial da intenção
 - ✓ Framework test-driven para LLMs
-
- 📊 Redução de 35-40% na carga cognitiva (NASA TLX)
 - 📊 Aumento de 110% na acurácia (0.40 → 0.84)
 - 📊 Equalização entre modelos grandes e pequenos
-
- 🔧 "Potencializa confiança e interpretabilidade"
 - 🔧 "Concilia IA e raciocínio humano"
 - 🔧 "Novo paradigma: eng. de prompt + eng. de teste"

Limitações

- ✗ Testes gerados podem estar incorretos
 - ✗ Complexidade cresce exponencialmente
 - ✗ Restrito a Python e funções isoladas
-
- 😬😞 Usuário pode validar erroneamente
 - 😬😞 Pressupõe literacia técnica sólida
 - 😬😞 Falta análise longitudinal (uso contínuo)
-
- 🔬 Tarefas simples e curtas (15min)
 - 🔬 Amostra pequena (n=15)
 - 🔬 Ignora aspectos sociais do desenvolvimento

O que vocês acham?

🤔 *"Testes gerados automaticamente substituem a compreensão humana ou a complementam?"*

💡 *"A ferramenta perfeita não é a que faz tudo, mas a que amplifica o melhor do humano"*



Muito
obrigado!



Centro de
Informática
UFPE